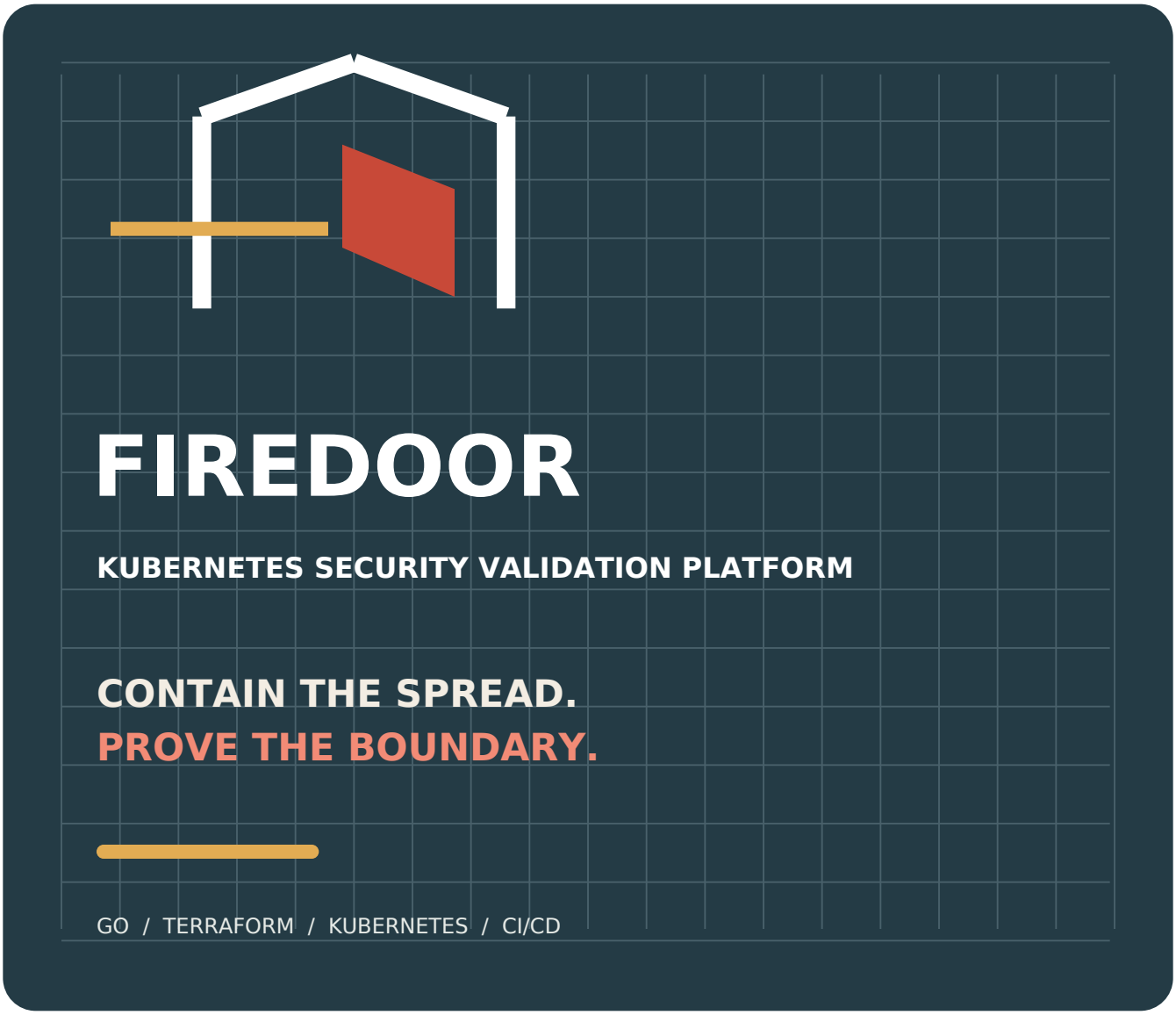


MASTER PROJECT MAP / STEP-BY-STEP BUILD GUIDE / V1.1

Definitive replacement of Firedoor v1.0: same product and safety constraints, redesigned as a Go-native Kubernetes verification engine with Terraform-managed security postures and CI-backed infrastructure proofs.

Planning baseline: 4 November 2026. All commands, directories and acceptance tests are proposed until implemented and verified on the chosen host. Current tool versions and upstream API semantics must be checked before coding.

00 / COMMAND BRIEF

How to use this guide

NORTH STAR

Firedoor verifies whether selected Kubernetes security controls actually contain an attempted compromise, and links observed behaviour to the intended Terraform configuration and Kubernetes evidence.

The five-step journey

DEPLOY	CHALLENGE	INSPECT	HARDEN	PROVE
Terraform applies an owned test profile	Go runs a bounded scenario	Collect Kubernetes and IaC evidence	Apply an approved profile change	Repeat and export a Boundary Proof

v1.0 success is not a screenshot

- Five safe, independently resettable scenarios FD-001 through FD-005; baseline/hardened comparisons.
- All 15 existing control requirements remain traceable; each applicable assertion has raw evidence.
- Go CLI and Go HTTP API, with a separate React console reading the same versioned run data.
- Terraform manages the long-lived testbed; cluster bootstrap is separate; controlled configuration drift is detectable.
- A fresh clone can bootstrap, validate, compare, export, and destroy the local lab without paid services.
- CI rejects a deliberately weakened Terraform policy and shows why; cleanup is reliably enforced.

Key portfolio checkpoints

Checkpoint	What must be demonstrable
v0.3	Real Go test of a Terraform-managed NetworkPolicy, before and after hardening.
v0.7	Containment Map from actual evidence with links to Terraform configuration.
v0.9	CI catches infrastructure security regression, publishes results, cleans up.
v1.0	All five scenarios, proof artifacts, reproducible setup, tests, polished launch.

01 / IDENTITY

Brand identity - unchanged

A physical fire door is designed to compartmentalize a building and limit spread. Firedoor asks the equivalent question of Kubernetes controls: does the configured boundary actually hold under a defined test?

Colour	HEX	Purpose
Porcelain	#F2EDE3	Light workspace
Structural slate	#243B45	Navigation and architectural diagrams
Fire red	#C84938	Product identity / primary emphasis
Inspection amber	#E2AC53	Caution / test in progress
Safety green	#4F9274	Verified defensive result
Concrete	#9AA0A6	Neutral / inactive

Typography and logo

- Barlow Condensed: headlines and scenario numbers. IBM Plex Sans: readable interface. IBM Plex Mono: commands, HCL, YAML, IDs and evidence.
- Original mark: geometric doorframe, inner barrier and interrupted traversal path. Produce wordmark, monogram, favicon, GitHub avatar, dark/light lockups and report header.
- Do not use generic flame/shield imagery. Avoid conflating fire red with failure; outcomes must use explicit text and accessible symbols.

Brand story strengthened by Terraform

TECHNICAL LINE Terraform declares. Kubernetes enforces. Go verifies. Firedoor proves.

The new story adds desired-state configuration and drift evidence without changing the product promise: Contain the spread. Prove the boundary.

02 / EXPERIENCE

Product vocabulary & feature contract

Module	User job	Hero detail
Overview	See environment readiness and recent validation history.	Testbed profile, state, last verified paths
Containment Map	Explore observed workload/namespace boundaries.	Click an edge -> proof, policy, attempted action
Scenarios	Choose one of five controlled validation contracts.	Baseline/hardened expected outcomes
Control Panel	Inspect settings and preview approved profile changes.	Terraform plan summary + Kubernetes snapshot
Proof Lens	Investigate exact evidence and control attribution.	HCL source, planned change, live objects, probe
Boundary Proofs	Export reviewable versioned validation artifacts.	Desired / observed / tested / limitations
Environment	Bootstrap and cleanly destroy owned infrastructure.	kind, CNI, Terraform state and drift health

Official statuses

Status	Precise meaning
Crossed	Test succeeded where scenario expected a restricted path.
Contained	Restriction verified under tested conditions with credible attribution.
Blocked	Low-level failure observation; not alone evidence of containment.
Inconclusive	Missing/ambiguous observation, attribution or environment.
Not tested	No completed qualifying validation.
Drift detected	Terraform actual vs desired differs; not equivalent to exploitable flaw.
NAMING RULE	Use real Kubernetes terms in technical views. Never rename RBAC, NetworkPolicy, ServiceAccount, Pod Security, Terraform plan, or Terraform state merely for brand consistency.

03 / ARCHITECTURE

System architecture & contract boundaries

Layer	Official technology	Responsibility
Bootstrap	kind + verified Calico/Cilium-compatible CNI	Create local cluster; install and prove network enforcement
Desired state	Terraform + Kubernetes provider	Manage long-lived workloads, namespace, identities and controls
Engine	Go + client-go	Read state, run controlled probes, evaluate security assertions
CLI	Go + Cobra	Orchestration, audit, validation, comparison, proof, lifecycle
Local API	Go net/http or Chi	Read evidence first; privileged actions behind explicit safeguards
Evidence	SQLite + versioned JSON	Immutable run provenance and proof artifacts
Console	React, TypeScript, Vite, React Flow	Containment Map and proof investigation; no duplicated verdict logic
Automation	GitHub Actions	Terraform checks, Go tests, ephemeral cluster regressions

Architecture principle

SEPARATE OWNERSHIP

kind creates cluster. Bootstrap installs CNI. Terraform owns declarative long-lived objects. Go owns only short-lived namespaced probes and the evidence store. No dual writers for the same resource.

Execution order

1. Verify host + expected kube context -> 2. kind create -> 3. install CNI -> 4. CNI enforcement smoke test -> 5. Terraform init/plan/apply -> 6. Go inspect/validate -> 7. capture evidence + compare -> 8. Terraform destroy -> 9. kind delete.

Do not build a Kubernetes operator in V1. A local CLI and explicit orchestration provides simpler and safer scope. Future reconcile loops are justified only by a real continuous-validation use case.

04 / RESOURCE OWNERSHIP

Bootstrap, Terraform and Go responsibilities

Object/action	Owner	Boundary
kind cluster create/delete	Bootstrap	Named, dedicated Firedoor cluster only
CNI installation / verification	Bootstrap	Pinned manifest and verified NetworkPolicy support
Namespaces / long-lived services	Terraform	Provider-managed state and resource labels
NetworkPolicies, RBAC, identities	Terraform	Never patched silently by Go
Pod security and workloads	Terraform	Profile-driven only; intentional drift tested separately
Short-lived validation pods/jobs	Go engine	Labelled, TTL-bound, confined to owned namespace
Evidence / verdict / exports	Go engine	Persisted separately from Terraform state
Plan preview / apply request	Go orchestrator	Executes approved Terraform process after scope/approval
CI environment	GitHub Actions	Unique ephemeral cluster and isolated Terraform state

Dual-writer problem and solution

- Go must not directly update Terraform-owned NetworkPolicy, RoleBinding or Deployment objects.
- For profile changes, CLI displays a Terraform plan, checks the target cluster fingerprint, and requires explicit confirmation before apply.
- For drift experiments, use a constrained helper against a single owned test resource; collect a snapshot; restore from Terraform and verify no residual drift.
- Terraform destroy happens before cluster deletion. Deleting the cluster first can strand Terraform state and make normal teardown fail.

STATE SECURITY Terraform state may contain resource details and secrets. Keep *.tfstate, *.tfvars with credentials, .terraform/ and plan files out of Git; minimize or avoid secrets entirely using synthetic data.

05 / BOOTSTRAP

Cluster topology and CNI prerequisite

Keep the original namespace model. edge: frontend workload. app: API and worker. data: database and cache. firedoor-system: owned validation support. All services and identities are fictitious, disposable and scoped.

Correct bootstrap sequence

```
# Planned commands; file and CNI choice verified in v0.1
kind create cluster --name firedoor-local \
  --config infrastructure/bootstrap/kind.yaml
kubectl --context kind-firedoor-local get nodes
# Install pinned CNI from reviewed instructions; verify ready.
kubectl --context kind-firedoor-local get pods -A
```

- Define kind with disableDefaultCNI when selecting an alternative; verify compatibility with Kubernetes and kind versions.
- Install a pinned compatible CNI after cluster creation; do not infer NetworkPolicy enforcement merely from objects existing.
- Smoke-test healthy destination, permitted source and denied source with DNS/port details before recording a containment verdict.
- Disable/defer Terraform apply until CNI readiness and independent allow/deny checks pass.

Environmental acceptance

Positive control	Negative control	Expected
Allowed test client to healthy test service	Denied test client to same healthy service	Allowed succeeds; denied fails after verified policy enforcement.
Probe without policy	Probe with known blocking policy	Observed difference is attributable, not an outage.

FAILURE POLICY If the destination is unhealthy, DNS broken, or the CNI cannot be verified, mark scenario INCONCLUSIVE. Never report Contained based only on a failed TCP connection.

06 / TERRAFORM

Terraform configuration blueprint

One authoritative testbed root

```
infrastructure/terraform/
  modules/
    workload/
    identity/
    network-boundary/
    pod-security/
  testbed/
    versions.tf
    providers.tf
    variables.tf
    main.tf
    outputs.tf
    tests/
      network_contract.tftest.hcl
      identity_contract.tftest.hcl
```

- Use provider version constraints and commit the dependency lockfile. Use a deliberate module API with typed inputs/outputs and documented defaults.
- Use a single testbed state and explicit security_profile = baseline | hardened input for the same logical managed resources. Do not use parallel states to manage the same objects.
- Use an explicitly selected kubeconfig and context for the provider. The Go orchestrator independently verifies cluster ID and Firedoor ownership marker before plan/apply.
- Manage namespace labels, Deployments, Services, ServiceAccounts, RBAC, NetworkPolicies, and supported Pod security policy/label definitions.
- Keep bootstrap and Terraform roots separate to avoid creation-time provider dependency cycles.

Terraform responsibilities by module

Module	Input contract	Outputs
workload	namespace, image tag/digest, resources	deployment/service identity
identity	SA name, allowed role rules	SA and binding identities
network-boundary	selectors, ports, profile	network policy references
pod-security	namespace labels, security context	applicable setting references

07 / DESIRED STATE

Profile changes, drift and state discipline

Safe profile transition - no competing resource owners

```
cd infrastructure/terraform/testbed
terraform init
terraform fmt -check -recursive
terraform validate
terraform plan -var="security_profile=baseline" -out=baseline.tfplan
# Explicit approval and context verification, then apply.
terraform apply baseline.tfplan
# Repeat plan/apply for hardened using SAME state, not two states.
```

- Never check generated plan files into the repository. Separate local shell environment details from copied/published examples.
- A baseline profile is intentionally permissive only for the specific controlled cases; no external exposure, real user data or unrestricted targets.
- After applying hardened profile, run a fresh plan to inspect unexpected drift; do not interpret a clean plan as proof of security.

Drift demonstration contract

Step	Required verification
1. Capture desired state	Record Terraform configuration commit, profile, plan summary, state lineage identifier.
2. Simulate a narrow out-of-band change	Only change a labelled Firedoor-owned disposable policy, under explicit test mode.
3. Observe drift	Terraform plan differs; capture changed object and actual field.
4. Validate behaviour	Go runs same bounded probe with health controls; record actual effect.
5. Restore	Terraform reapplies intended state; Go verifies behaviour restored; no leftover drift.

CRITICAL DISTINCTION Drift detected is not automatically a vulnerability. A failed or unexpected validation can be unrelated to drift; record and compare both rather than inventing a causal link.

08 / CONTROLS

Fifteen controls: original coverage retained

Legacy	Required control	Primary scenario/domain
C-001	Dedicated workload ServiceAccount	Identity
C-002	Disable unnecessary SA token automount	Identity
C-003	Least-privilege Kubernetes RBAC	Identity
C-004	Default-deny network profile	Network
C-005	Required ingress/egress exceptions	Network
C-006	Run as non-root	Workload
C-007	Disable privilege escalation	Workload
C-008	Disallow privileged containers	Admission
C-009	Minimize Linux capabilities	Workload
C-010	Read-only root FS where compatible	Workload
C-011	Pod Security labels/standards	Admission
C-012	Appropriate seccomp profile	Workload
C-013	Narrow secret permissions and mounts	Secrets
C-014	Resource requests and limits	Resilience
C-015	Pinned/controlled images and optional scan evidence	Supply chain

Maintain C-001 through C-015 as legacy references and introduce stable Firedoor control IDs such as CTRL-NET-001. Include a mapping table in docs/control-migration.md. Preserve the original policy objectives even as HCL structure evolves.

CONTROL VERDICT

Each Go evaluator records applicability, current Kubernetes observation, Terraform declaration reference, expected policy, relevant evidence, and limitations. Configured control ≠ verified behavioural enforcement.

09 / SCENARIO CONTRACTS

Five bounded, reproducible scenarios

ID	Name	Expected security claim
FD-001	Compromised Workload	A synthetic foothold in frontend has a defined restricted reach.
FD-002	Identity Overreach	Workload SA cannot perform forbidden Kubernetes API actions.
FD-003	Secret Boundary	Unauthorized SA cannot read an unrelated synthetic secret.
FD-004	Lateral Movement	edge/front-end cannot directly reach protected data service in hardened profile.
FD-005	Privileged Workload Admission	Unsafe workload security settings are rejected by the configured admission policy.

Every scenario package must specify

- Stable ID/schema version; owner and purpose; approved cluster/context scope; fixture inputs; exact Kubernetes target and operation.
- Required baseline and hardened Terraform profile conditions; expected observations; mapped controls and reviewed ATT&CK; metadata where justified.
- Preflight checks (destination health, permission scope, CNI readiness); timeouts; cancellation; cleanup; isolated run data and hash-referenced evidence.
- Negative controls, explicit inconclusive/error cases, independent test reruns and CI integration.

FD-004 first - a precise security proof

Test frontend -> data service under baseline, then apply a Terraform hardening change in the same owned state, verify all targets healthy, rerun the same probe, and attribute the denial to the intended NetworkPolicy only if independent allowed/blocked checks support it.

NO OVERCLAIM FD-005 is a pod admission/security-controls test, not a real container breakout or kernel exploit. Scenarios must not target production clusters or harvest real credentials.

10 / EVIDENCE

Infrastructure Proof: the new differentiator

The original Boundary Proof is retained and extended with Infrastructure Proof fields. These connect infrastructure-as-code intent to observed resource state and tested behaviour, without treating any one layer as sufficient evidence.

Evidence layer	Recorded facts
Declared state	Git commit, Terraform module/profile, plan summary/hash, provider lock/version.
Observed state	Cluster fingerprint, resource metadata/UID/generation, actual policy snapshot, applied labels.
Behavioural test	Scenario version, src/dst/port or API request, health controls, observed result, timing.
Attribution	Policy and control references, positive/negative controls, ambiguity/confidence rationale.
Proof artifact	Run IDs, timestamps, hashes, verdict, limitations, source artifact references.

Canonical versioned objects

```
Scenario: id, schema_version, steps, mapped_controls, expectations
Run: run_id, scenario, git_sha, profile, cluster_fingerprint,
     terraform_plan_hash, started_at, step_results, verdict
Evidence: evidence_id, kind, source, target, observation, artifact_sha256
Proof: proof_id, run_refs, declaration, observed_state,
       tested_claim, attribution, confidence, limitations
```

- Persist JSON artifacts and a searchable SQLite index. All API/CLI/console views must share the Go verdict implementation.
- Record both declared HCL and live Kubernetes snapshots; do not store sensitive Terraform state or raw auth tokens in public proof bundles.
- Contradictory or missing observations downgrade the claim to Inconclusive.

11 / REPOSITORY

Planned Go + Terraform repository layout

```
firedoor/
  README.md  LICENSE  SECURITY.md  CONTRIBUTING.md  go.mod  go.sum
  cmd/firedoor/                # Cobra CLI entry point
  internal/
    cluster/                   # scope, kube client and discovery
    controls/                  # typed evaluators
    validation/                # bounded scenario runner
    profiles/                  # Terraform plan/apply orchestration
    evidence/                  # capture, hashing, SQLite/JSON
    reporting/                 # Boundary / Infrastructure Proofs
    api/                       # Go HTTP handlers
  pkg/models/                  # versioned public JSON contracts
  infrastructure/
    bootstrap/                 # kind and CNI manifests/scripts
    terraform/
      modules/{workload,identity,network-boundary,pod-security}/
      testbed/                 # one root and one authoritative state
  apps/console/                # React + TypeScript + React Flow
  scenarios/{FD-001,FD-002,FD-003,FD-004,FD-005}/
  tests/{integration,scenario}/
  docs/{adr,scenarios,terraform,controls,demos}/
  design/{brand,tokens}/
  .github/workflows/
```

Create only the folders needed at the current milestone; reserve names now so the roadmap remains stable. Avoid starting with needless multiple Go services. A modular monolith with a CLI and small HTTP endpoint is enough.

IDs and data contracts

Identifier	Example
Scenario	FD-004
Validation	VAL-000042
Control	CTRL-NET-001
Evidence	EV-000128
Proof	PROOF-000042
Kubernetes context	kind-firedoor-local
Terraform root	infrastructure/terraform/testbed

12 / CLI CONTRACT

Command surface and happy-path workflow

```
# Target CLI contracts; not implemented yet.
firedoor doctor
firedoor setup
firedoor status
firedoor environment plan --profile baseline
firedoor environment apply --profile baseline
firedoor audit --format json
firedoor validate FD-004 --profile baseline
firedoor environment plan --profile hardened
firedoor environment apply --profile hardened
firedoor validate FD-004 --profile hardened
firedoor compare FD-004
firedoor drift check
firedoor proof export VAL-000042 --format html
firedoor reset
firedoor destroy
```

CLI design requirements

- Commands have --help, stable exit codes, structured JSON, readable human output, bounded timeouts and cancellation.
- Terraform plan/apply must not be silently bypassed; require both approved profile, live cluster identity, and destructive-action confirmation.
- Use direct executable invocation rather than fragile shell-string composition; pin approved root/path, variables and binary version.
- Surface state lock/lease failure, absent CNI, drift, unsupported scenarios and missing permissions as explicit error types.
- Global --context may be read-only for diagnostics, but privileged lab actions in v1 are restricted to owned cluster.

SAFETY DEFAULT Dry-run before changes; no automatic apply, cleanup or external cluster mutation from opening a UI page. Proposed CLI commands become real only when their associated milestone tests pass.

13 / ROADMAP

Version map and release gates

Version	Core delivery	Go + Terraform + K8s proof
v0.0	Foundation	Go module/Cobra, Terraform skeleton, CI and contracts
v0.1	Testbed	kind/CNI verified; Terraform applies owned workloads
v0.2	Controls	Go evaluators cross-reference IaC and live objects
v0.3	First path proof	FD-004 crosses baseline and is contained after Terraform hardening
v0.4	Scenario library	All five bounded security claims validated
v0.5	Proof Lens	Persistent evidence and IaC provenance
v0.6	Console	Go HTTP API + real-data React UI
v0.7	Containment Map	Observed edge -> Kubernetes and HCL evidence
v0.8	Boundary Proofs	Export Infrastructure Proof and baseline/hardened comparison
v0.9	CI regression	Terraform checks + kind + Go security regression + cleanup
v1.0	Flagship release	Fresh clone reproduces all five tests and final demo

Release discipline

- One verifiable vertical slice per PR; run smoke tests before claiming a milestone complete.
- At the end of every slice, preserve a progress note with exact commands, commit, observed evidence, unresolved risks and the next first task.
- The roadmap is sequential; tests and documented exit gates matter more than a calendar date.

NO FEATURE CREEP

Do not require AKS, Falco, policy engines, GitOps controllers or remote state before v1. These are optional post-v1 extensions, not prerequisites.

MILESTONE / V0.0

v0.0 - Foundation - Go and IaC blueprint

Branch: feat/v00-foundation | Outcome: Foundation - Go and IaC blueprint

Implementation slices

- 01. Create Git SSH repository, README, license, SECURITY.md, CONTRIBUTING.md and first ADRs; verify name prior to public launch.
- 02. Initialize Go module, Cobra root command, version/doctor placeholders, gofmt/go vet/go test and CI.
- 03. Create Terraform module/testbed directories; document version pinning, scope limits, state ignore rules and one-root profile model.
- 04. Draft Go evidence contracts, all five scenario specs, architecture/ownership diagram and brand tokens.
- 05. Add early React wireframes without creating fake product data.

Definition of done - required gates

- Go toolchain builds and unit tests run; Cobra --help works.
- Terraform fmt and validate work on a minimal provider configuration (without cluster apply).
- First CI job green and scenario contracts/evidence schemas documented.

Target verification - proposed commands

```
go version
go test ./...
go vet ./...
go run ./cmd/firedoor --help
terraform -chdir=infrastructure/terraform/testbed fmt -check -recursive
terraform -chdir=infrastructure/terraform/testbed init -backend=false
terraform -chdir=infrastructure/terraform/testbed validate
```

RISK / CAUTION Provider init may need internet; Terraform validation with cluster-dependent resources may require additional scoping. No cluster mutation permitted in v0.0.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Go toolchain, Cobra, Terraform module structure, Git/CI.

MILESTONE / V0.1

v0.1 - Environment - Terraform testbed

Branch: feat/v01-environment | Outcome: Environment - Terraform testbed

Implementation slices

- 01. Create kind cluster config and unique Firedoor ownership marker; implement Go context and fingerprint checks.
- 02. Install reviewed CNI with pinned manifest/version; verify independent NetworkPolicy allow/deny, health and DNS controls.
- 03. Build Terraform testbed root and first reusable modules for namespaces, workloads, identities, policy.
- 04. Implement baseline/hardened security_profile variable and one local authoritative state.
- 05. Add Go setup/status/destroy orchestrator; teardown must destroy Terraform resources before deleting kind.

Definition of done - required gates

- Fresh cluster creation produces working edge/app/data services.
- Terraform plan/apply targets owned kind context; second plan shows no unexplained drift.
- Independent CNI enforcement proven; repeated lifecycle succeeds; no leftovers after destroy.

Target verification - proposed commands

```
kind create cluster --name firedoor-local --config infrastructure/bootstrap/kind.yaml
kubectl --context kind-firedoor-local get pods -A
terraform -chdir=infrastructure/terraform/testbed init
terraform -chdir=infrastructure/terraform/testbed plan -var=security_profile=baseline
firedoor status
firedoor destroy
```

RISK / CAUTION No network-policy verdict until CNI enforcement test is independently verified. Do not delete kind before Terraform destroy.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Terraform HCL, provider config, kubectl, kind/CNI, Go client-go.

MILESTONE / V0.2

v0.2 - Control Panel - inspected security state

Branch: feat/v02-controls | Outcome: Control Panel - inspected security state

Implementation slices

- 01. Define Go control evaluator interfaces, applicability, severity, evidence and verdict contracts.
- 02. Implement six foundational checks: SA, token automount, RBAC, default-deny network, non-root, privilege escalation.
- 03. Link each Go finding to observed Kubernetes resource and its Terraform module/profile reference.
- 04. Implement repeatable CLI audit and transparent posture scores with Not Applicable handling.
- 05. Add unit tests for malformed objects, missing policies, invalid context, and drifted observed state.

Definition of done - required gates

- Expected baseline/hardened differences are explainable, not fabricated.
- Finding IDs map to original C-001...C-015 and structured evidence.
- Go unit/integration tests green; unknown context fails closed.

Target verification - proposed commands

```
go test ./internal/controls/... ./internal/cluster/...
firedoor audit --profile baseline --format json
firedoor audit --profile hardened
terraform -chdir=infrastructure/terraform/testbed validate
```

RISK / CAUTION Static presence of a NetworkPolicy does not prove enforcement; defer behavioural containment verdict to scenario validation.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Go interfaces, typed Kubernetes APIs, RBAC, Terraform resource references.

MILESTONE / V0.3

v0.3 - Validation engine - first boundary proof

Branch: feat/v03-first-validation | Outcome: Validation engine - first boundary proof

Implementation slices

- 01. Implement bounded FD-004 Go probe using owned frontend identity, protected data service and timeout-aware checks.
- 02. Run baseline profile test; capture allowed connection, target health, DNS and object snapshots.
- 03. Generate and review hardened Terraform plan; apply under same state, verify policy and cluster identity.
- 04. Repeat identical probe with independent allowed-path control; establish policy attribution or mark inconclusive.
- 05. Persist before/after comparison and exact resource/configuration references; ensure repeatable reset.

Definition of done - required gates

- Baseline reaches protected service as expected; hardened profile restricts the exact tested path.
- Positive controls and policy snapshots support causal attribution.
- Fresh owned testbed reproduces comparison; Go integration tests pass.

Target verification - proposed commands

```
firedoor environment plan --profile baseline
firedoor validate FD-004 --profile baseline
firedoor environment plan --profile hardened
firedoor environment apply --profile hardened
firedoor validate FD-004 --profile hardened
firedoor compare FD-004
```

RISK / CAUTION Do not treat timeout/DNS or unhealthy service as NetworkPolicy proof. State/profile transitions must be explicit and sequential.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Go cluster probes, Terraform plan/apply, Kubernetes NetworkPolicy, CI-ready evidence.

MILESTONE / V0.4

v0.4 - Scenario library - complete bounded coverage

Branch: feat/v04-scenarios | Outcome: Scenario library - complete bounded coverage

Implementation slices

- 01. Implement FD-001 workload foothold boundaries with synthetic data and valid assumptions.
- 02. Implement FD-002 API RBAC/SA overreach, recording precise Kubernetes API authorization outcomes.
- 03. Implement FD-003 synthetic secret boundary without reading real secret content.
- 04. Implement FD-005 privileged workload admission controls, capturing rejected admission responses.
- 05. Ensure every scenario uses approved Terraform profiles, Go runner isolation, cleanup and evidence contracts.

Definition of done - required gates

- All five scenarios produce expected baseline/hardened evidence and relevant inconclusive states.
- Tests are independent of execution order and restore state.
- No arbitrary target scanning or unrestricted exploitation support.

Target verification - proposed commands

```
firedoor scenarios list
firedoor validate FD-002 --profile hardened
firedoor validate FD-003 --profile hardened
firedoor reset
go test ./internal/validation/... ./tests/...
```

RISK / CAUTION FD-005 validates admission and security settings, not a real container escape. Synthetic test objects must be isolated and cleaned up.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Go scenario engine, Kubernetes auth/admission/secrets, Terraform security profiles.

MILESTONE / V0.5

v0.5 - Proof Lens - evidence and IaC provenance

Branch: feat/v05-evidence | Outcome: Proof Lens - evidence and IaC provenance

Implementation slices

- 01. Build versioned Run/Evidence/Proof contracts with JSON, SQLite indexes and integrity hashes.
- 02. Capture Terraform plan metadata and module references without leaking state or sensitive variables.
- 03. Capture live Kubernetes objects, request details, expected/actual outcomes, health controls and attribution.
- 04. Provide Go read-only API/CLI detail and comparison endpoints; document migration strategy.
- 05. Test tampered, incomplete and mismatched evidence as explicit inconclusive outcomes.

Definition of done - required gates

- An independent reviewer can explain one FD-004 verdict from artifacts without UI.
- Desired state, observed state and behavioural test are visibly separated.
- Evidence schema migrations and integrity checks pass.

Target verification - proposed commands

```
go test ./internal/evidence/... ./internal/reporting/...
firedoor compare FD-004
firedoor proof show VAL-000042
firedoor audit --format json
```

RISK / CAUTION Never publish terraform.tfstate or credential-bearing plan files; exports include sanitized plan summaries and hashes only.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Go serialization, SQLite, provenance, Terraform and Kubernetes evidence.

MILESTONE / V0.6

v0.6 - Firedoor Console - Go API + React

Branch: feat/v06-console | Outcome: Firedoor Console - Go API + React

Implementation slices

- 01. Implement read-only Go HTTP endpoints for environment, controls, scenarios, runs, proof and drift summary.
- 02. Build branded React Overview, Scenarios, Control Panel, Environment and run history screens.
- 03. Ensure all UI verdicts, status and control scores originate from Go engine persisted data.
- 04. Add explicit loading/empty/error states, accessibility and responsive layouts.
- 05. Add contract tests for Go API -> TypeScript interface and frontend build/test checks.

Definition of done - required gates

- UI opens genuine FD-004 evidence and Terraform/Kubernetes references.
- No fabricated production-like results or independent frontend scoring.
- Build passes at laptop and 1440px; API is locally secured for sensitive actions.

Target verification - proposed commands

```
go test ./internal/api/... ./...
npm --prefix apps/console run build
firedoor status
```

RISK / CAUTION No unauthenticated HTTP endpoint may allow arbitrary Terraform apply, destroy or cluster mutation. Start read-only.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Go HTTP API, React/TypeScript, API contracts and authentication boundaries.

MILESTONE / V0.7

v0.7 - Containment Map - flagship visual

Branch: feat/v07-containment-map | Outcome: Containment Map - flagship visual

Implementation slices

- 01. Model namespaces, workload nodes, network edges and observed probe results using actual Go evidence.
- 02. Create React Flow map with boundary zones, successful crossings, blocked edges, untested and inconclusive states.
- 03. Add edge-click Proof Lens drawer showing policy, Terraform module reference, and underlying test.
- 04. Add baseline/hardened comparison overlay and labels for declared vs observed configuration.
- 05. Capture a real tested FD-004 screenshot and ≤90-second preview with documented data provenance.

Definition of done - required gates

- Every observed path traces to real run and evidence.
- Labels do not confuse hypothetical topology with tested reachability.
- Readable and keyboard accessible at demo resolutions.

Target verification - proposed commands

```
firedoor validate FD-004 --profile baseline
firedoor validate FD-004 --profile hardened
npm --prefix apps/console run build
go test ./internal/api/...
```

RISK / CAUTION	Do not infer arbitrary reachable paths from Kubernetes service existence; only draw tested results as verified edges.
CLEAN STOP	Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: React Flow, Go APIs, Kubernetes topology, Terraform evidence presentation.

MILESTONE / V0.8

v0.8 - Boundary Proofs - Infrastructure Proof

Branch: feat/v08-proofs | Outcome: Boundary Proofs - Infrastructure Proof

Implementation slices

- 01. Generate branded HTML and JSON Boundary Proof exports using persisted Go data.
- 02. Add Infrastructure Proof: HCL commit/profile, sanitized plan summary, observed Kubernetes policy, behavioural outcome.
- 03. Produce baseline/hardened side-by-side reports with evidence links and limitations.
- 04. Add safe escaping and report reproducibility tests; avoid embedding secrets/terraform state.
- 05. Introduce controlled drift test contract and proof representation independent of exploitability claims.

Definition of done - required gates

- One CLI command reopens and exports past real evidence.
- Reports separate intended, observed and tested assertions and match CLI/console.
- Plan references, hashes, timestamps and schema version are independently verifiable.

Target verification - proposed commands

```
firedoor proof export VAL-000042 --format html
firedoor proof export VAL-000042 --format json
go test ./internal/reporting/... ./internal/evidence/...
```

RISK / CAUTION	A clean Terraform plan is only a desired-state comparison, not a security certificate. Avoid storing full sensitive state.
CLEAN STOP	Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Terraform desired-state evidence, audit reporting, Go HTML/JSON export.

MILESTONE / V0.9

v0.9 - Boundary regression - CI and drift

Branch: feat/v09-regression | Outcome: Boundary regression - CI and drift

Implementation slices

- 01. Build GitHub workflow with Go lint/vet/test, Terraform fmt/validate/tests, console build and ephemeral kind suite.
- 02. Each integration job creates a uniquely named disposable cluster, installs verified CNI, applies testbed Terraform.
- 03. Execute hardened FD-004 and other representative scenario checks; publish JSON/HTML artifacts.
- 04. Inject deliberately weakened Terraform NetworkPolicy fixture; ensure regression is detected and fixed version passes.
- 05. Add a controlled drift demonstration: detect out-of-band mutation, validate actual effect, restore desired state.
- 06. Ensure Terraform destroy and kind deletion run on both passing and failing jobs; verify no leftovers.

Definition of done - required gates

- Deliberately weakened security policy yields failed required CI check with evidence.
- Approved repaired policy passes same suite.
- No cloud creds on untrusted PR; every CI run cleans up tracked resources.

Target verification - proposed commands

```
go test ./...
go vet ./...
terraform -chdir=infrastructure/terraform/testbed fmt -check -recursive
terraform -chdir=infrastructure/terraform/testbed validate
terraform -chdir=infrastructure/terraform/testbed test
# GitHub Actions performs full ephemeral kind + scenario suite.
```

RISK / CAUTION Terraform tests may apply real resources. Constrain them to dedicated ephemeral clusters and verify teardown even on failure; never target arbitrary kube context.

CLEAN STOP Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Go CI, Terraform tests/drift, Kubernetes security regression, GitHub Actions.

MILESTONE / V1.0

v1.0 - Public flagship release

Branch: release/v1.0-rc | Outcome: Public flagship release

Implementation slices

- 01. Run a fresh-clone local bootstrap, Terraform apply, all five scenarios, proof exports, destroy and clean state verification.
- 02. Finish dependency review, threat model, state/secrets audit, documentation, accessible UI and reliability testing.
- 03. Finalize logo, screenshots, README narrative, recorded demo, architecture, contributor and security docs.
- 04. Publish tagged release with changelog, CI status, executable/tested usage steps and limitations.
- 05. Ensure reviewer can reproduce core value without Azure or production credentials.

Definition of done - required gates

- Go CLI + Terraform + kind/CNI + React integration passes accepted tests.
- FD-001 through FD-005 are repeatable and full cleanup succeeds.
- All GitHub screenshots/reports use genuine run results and scope-limited claims.

Target verification - proposed commands

```
firedoor doctor
firedoor setup
firedoor environment apply --profile baseline
firedoor validate FD-004 --profile baseline
firedoor environment apply --profile hardened
firedoor validate FD-004 --profile hardened
firedoor compare FD-004
firedoor proof export VAL-000042 --format html
firedoor destroy
```

RISK / CAUTION	Do not release while scope guards, Terraform state lifecycle or lab cleanup are unresolved. Re-test from a clean environment.
CLEAN STOP	Review the diff; record versions, command outputs and test evidence in docs/progress.md; commit and push a scoped PR; merge only after gates pass; confirm a clean main branch and write the exact first action for the next milestone.

Career proof: Production-style Go, Terraform and Kubernetes engineering with verifiable security demonstration.

14 / CI DETAIL

GitHub Actions pipeline - exact engineering gates

Job	Check	Failure condition
go-quality	gofmt; go vet; go test -race where compatible	Broken types, failed tests, races
terraform-quality	fmt, init -backend=false, validate, targeted terraform test	Invalid HCL / failed module assertions
console-quality	npm ci; lint/test/build	Broken UI contracts or build
kind-security-regression	Bootstrap -> CNI test -> Terraform apply -> FD-004 -> compare	Unexpected accessible hardened path or missing evidence
negative-control	Deliberately weakened Terraform fixture	Must fail Go security gate, not pass
cleanup	Terraform destroy then kind delete, verify remaining clusters	Leaked owned resources or state ambiguity

Trust and permissions

- Untrusted PR jobs receive no Azure tokens, remote Terraform backend secrets or reusable production kubeconfig.
- Each job uses a disposable cluster name unique to its run; verify ownership marker and context before execution.
- Use pinned actions and provider lockfile; upload evidence artifacts with sensitive fields filtered.
- Always cleanup on failure/cancel where technically possible, and document manual cleanup when the runner is interrupted.

Quality demonstration

The signature GitHub demo changes a Terraform network restriction, observes a required scenario fail, then fixes HCL and shows a passing PR. Reviewer can inspect Go test evidence, Terraform plan summary and cluster snapshots from the same run.

IMPORTANT A GitHub check blocks merging only if required branch protection or an equivalent policy is actually configured. Do not claim enforcement from a green/red optional workflow alone.

15 / SAFETY

Threat model, state protection & failure policy

- Target restriction: only local Firedoor-owned kind context; verify cluster UID/fingerprint, namespace labels and explicit owner marker before any mutation.
- Terraform state: dedicated local root, state excluded from Git, no production secrets or private cluster credentials in evidence. State locking and sensitive redaction must be addressed.
- No cross-scope execution: arbitrary kubeconfig paths or --context overrides cannot bypass mutate guard; no unrestricted network scanning, credential theft or unsanctioned persistence.
- Destructive operations: preview plan + explicit acknowledgement; avoid ambient default-context changes and unexpected prompt-free apply/destroy.
- Profiles: baseline intentionally permissive only inside disposable lab and never external-facing.
- Probe restrictions: namespace-scoped Job/Pod, minimal SA, synthetic target, timeout/cancellation, cleanup on partial error.
- Verdict truthfulness: if cluster/CNI/target health unverified, verdict is inconclusive rather than proof of containment.

Threat model table

Threat	Control
Accidental production targeting	Identity guard + dedicated context + resource ownership marker.
Terraform and Go mutate same object	Single owner rule; Go orchestrates approved Terraform only.
Secret leak through logs/state	Synthetic secrets; redact exports; never commit state.
False positive containment claim	CNI smoke test, healthy target and independent allowed path.
Broken cancellation/partial apply	Timeout, rollback procedure, explicit residual resource inventory.
CI leaves infrastructure behind	finally-style destroy, cluster inventory and evidence on failed teardown.

16 / SKILL-BUILDING

Career matrix & engineering learning outcomes

Technology	Demonstrated competence	Portfolio evidence
Go	Typed models/interfaces, errors, HTTP, concurrency where needed, CLI	Unit tests, code architecture, Cobra commands
client-go	Read/authorize/launch bounded Kubernetes resources	Actual API calls, mocks, integration tests
Terraform	Modules, variables, outputs, providers, version lock, state, plan/apply	HCL modules, plan screenshots, test results
Kubernetes	ServiceAccounts, RBAC, NetworkPolicy, Pod Security, workloads	FD-001...FD-005 evidence
kind + CNI	Cluster lifecycle and real network policy enforcement	Positive/negative connectivity smoke tests
GitHub Actions	CI orchestration, protected checks, cleanup	Broken HCL regression PR and repaired PR
React / TypeScript	Professional evidence-focused Containment Map UI	Real-data responsive screenshots
Security engineering	Threat model, attack assumptions, provenance, bounded tests	Boundary Proofs and report limitations

Resume and interview differentiation

Suggested truthful result after completion: Built a Go-based Kubernetes security validation platform with Terraform-managed baseline/hardened controls; implemented policy/RBAC scenario tests, evidence-backed containment maps, and CI regression checks for infrastructure-as-code changes.

Do not claim all these technologies are implemented until the matching milestone and verification gate passes. A project pinned at v0.3 should state exactly what is complete.

17 / LAUNCH

GitHub, screenshots and demo story

The 75-second flagship demonstration

Time	Screen or evidence
00-08s	Product promise: Terraform declares / Kubernetes enforces / Go verifies.
08-19s	Terraform baseline profile; FD-004 connection reaches protected data tier.
19-30s	Proof Lens shows live objects, intended configuration and test evidence.
30-43s	Review Terraform hardening plan; apply approved NetworkPolicy.
43-56s	Same Go probe is blocked; independent control verifies policy attribution.
56-67s	Containment Map comparison and Infrastructure Proof.
67-75s	CI regression check, Go CLI, Terraform modules, GitHub end card.

README narrative

- Original Firedoor wordmark, one-line positioning and main Containment Map screenshot.
- Problem: a policy existing in HCL does not imply it was deployed or enforces the expected boundary.
- One-command local Quick Start with prerequisites and explicit cleanup.
- The five scenarios, 15 controls, architecture ownership model and exact evidence limitations.
- CI regression demonstration: weak NetworkPolicy rejected, corrected configuration passes.
- Screenshots from actual runs; badge and release notes; safety model and source references.

Proposed GitHub description

PUBLIC COPY

Crash-test Kubernetes security boundaries. Define controls with Terraform, verify them with Go, and prove containment through reproducible local scenarios and CI evidence.

18 / ADVANCED WORK

Future extensions - defer beyond 1.0

Candidate	Value	Why defer
AKS cloud profile	Terraform Azure provider, managed cluster, remote state, cloud networking	Cost/credential/deletion complexity
OPA/Gatekeeper/Kyverno	Richer admission policy testing and policy-as-code	Adds new enforcement semantics and controller lifecycle
Falco telemetry	Runtime visibility and event evidence	Optional dependency and resource cost
Go operator/controller	Continuous reconciliation of policies/validation cadence	Needs real operational use case
Remote Terraform state	Team collaboration and locking	Cloud dependency undermines local-first v1
Threshold integration	Kubernetes validation events -> detection tests -> Sentinel incident	Separate project must remain independently valuable

If we add AKS

- Create a separate Terraform root for cluster provisioning; do not mix managed cluster creation and in-cluster objects in one provider dependency chain.
- Require cost budget, time-boxed lab, remote-state design, scoped Azure identity, teardown verification and documentation.
- Firedoor remains local-first and retains same evidence schema; cloud tests add a distinct environment provenance tier.

SCOPE RULE No AKS, remote state, operator or cloud telemetry feature may block the local five-scenario v1.0 completion.

19 / HANDOFF

First session, quality checks & references

First development session - v0.0 only

- Record OS and versions of Go, Terraform, Docker, kind, kubectl, Node, Git, and CPU/RAM capacity.
- Confirm GitHub SSH key, inspect any existing Firedoor/BreachLab code, select repository name and check name collision risk.
- Create Go module, Cobra CLI scaffolding and first passing unit test; add Terraform directory/provider constraints.
- Write ADR-001 (Go engine), ADR-002 (Terraform ownership/state), ADR-003 (kind/CNI and safety scope).
- Create five scenario contracts and README. Open feature PR; do not build UI or mutate Kubernetes yet.

Repeatable stopping ritual

```
git status --short --branch
go test ./...
go vet ./...
git diff --check
git log -1 --oneline
git remote -v
# Terraform checks are run for active Terraform slices.
```

Current documentation to recheck when implementing

HashiCorp Kubernetes provider: https://developer.hashicorp.com/terraform/tutorials/kubernetes/kubernetes-provider

HashiCorp terraform test: https://developer.hashicorp.com/terraform/cli/commands/test

kind configuration/CNI: https://kind.sigs.k8s.io/docs/user/configuration/

Kubernetes official API/client-go: https://github.com/kubernetes/client-go

Terraform/Kubernetes provider releases, chosen CNI install guides, Go/Cobra and GitHub Actions documentation must be version-reviewed at implementation time.

SUPERSEDES

This v1.1 guide replaces the uploaded 25-page Firedoor v1.0 plan as the primary build reference. The brand, five scenarios, original 15 controls, and v0.0-v1.0 milestones remain; Python/Typer/FastAPI/pytest-specific architecture and commands are retired.